Optimized Gomoku AI Game Engine: System Architecture, Evidence Control, and 10-Week Research Backbone

Academic Institution: VNUK Institute for Research and Executive Education **Degree Module:** Basic Project (■■ Án C■ S■) — Software Engineering & Artificial Intelligence Track **Authoritative Repository:** [GitHub – tducn110/caro](https://github.com/tducn110/caro) **Local Repository Checkout:** `/home/pro/Downloads/vnuk/basicProject/10_caro` (Branch: `main`, HEAD: `cb6fd6f`) **Document Standard:** Standard 00 (IMRaD & Evidence Specification for Software Systems) **Control Layer Reference:** `REG-MASTER-00A`, `SYN-THESIS-98`, `REF-INDEX-99` **Primary Language:** English **Artifact Formats:** Markdown (`.md`), Microsoft Word (`.docx`), Adobe PDF (`.pdf`)

---

1. Repository Provenance & Authority Separation

The project repository `tducn110/caro` is the sole authoritative implementation source of truth for this project.

- **Repository Authority:** Confirms what has actually been implemented, tested, and committed.
- **Research Log Authority:** Explains why systems were designed, modeled, benchmarked, and interpreted.
- **Evidence Chain Invariant:** Specifications and reference algorithms remain labeled as **`[D]`** `Recommendation / Reference Only` until verified directly against repository code, test execution, and git commit hashes (`cb6fd6f`).

```
flowchart TD
    Spec["Specification & Architecture"] --> Repo["Repository Implementation (tducn110/caro)"]
    Repo --> Tests["Automated Test Suites (Vitest & Cargo)"]
    Tests --> Commit["Git Commit Hash / Tag (cb6fd6f)"]
    Commit --> Raw["Experimental Raw Data (CSV / JSON)"]
    Raw --> Charts["Tables & Visualizations"]
    Charts --> Claim["Thesis Scientific Claim ([A] Evidence)"]
```

---

2. Master End-to-End System Architecture

The architecture decouples UI rendering (React 18), 2D hardware graphics (PixiJS v8 WebGL), background concurrency (Web Workers), and high-throughput lookahead search (Rust WebAssembly):

```
flowchart TD
    subgraph UI_Layer ["Presentation & UI Layer (React 18 + HTML5 HUD)"]
        A1["Header & Difficulty Selector (Bot Tiers 1-6)"]
        A2["Turn Indicator, Move Clock & Scoreboard"]
        A3["Game Controls (New Game, Reset, Undo, Sound Toggle)"]
        A4["Game Over Modal & Victory Analysis Overlay"]
    end

    subgraph Canvas_Layer ["Hardware Canvas Rendering Layer (PixiJS v8 WebGL)"]
        B1["Viewport & Letterbox Scaler (600x600 px Design Space)"]
        B2["Board Grid Graphics (15x15 Lines & Star Points)"]
        B3["Stone Sprites Container (Batch-rendered 60 FPS)"]
        B4["Hover Indicator & Last-Move Pulsing Ring"]
        B5["Winning Five Laser Beam Highlight"]
    end

    subgraph Concurrency_Layer ["Thread Concurrency Boundary (Web Worker API)"]
        C1["Main UI Thread (Event Loop & DOM/Canvas Management)"]
        C2["Worker Message Bridge (postMessage / Transferable ArrayBuffer)"]
        C3["AI Worker Thread (Isolated Background Context - Zero Main Thread Blocking)"]
    end

    subgraph Bridge_Layer ["WASM Bridge & Memory Buffer Layer (TypeScript / wasm-bindgen)"]
        D1["CaroEngine Controller (TypeScript Ground Truth for PvP)"]
        D2["Linear Memory Buffer (Shared [u8; 225] Board State)"]
        D3["WASM Function Invoker (compute_best_move, evaluate_board)"]
    end

    subgraph Rust_Engine ["Core Rust Game & AI Engine (caro-core-reference)"]
        subgraph State_Model ["Domain State & Rules"]
... [truncated for print length] ...
```

---

3. System Thinking Framework

$$\text{INPUT} \longrightarrow \text{PROCESS} \longrightarrow \text{STATE} \longrightarrow \text{EVENT} \longrightarrow \text{SIDE EFFECT} \longrightarrow \text{OUTPUT} \longrightarrow \text{FEEDBACK}$$

3.1 Component Ownership Matrix

Subsystem Component	State Owner	Lifecycle Owner	Business Rule Owner	Render Owner	Concurrency Owner
Board State (`[u8; 225]`)	`CaroEngine` **(Rust Core)**	Game Session	`CaroEngine`	None (Data only)	Web Worker / Engine
Game Flow & Mode (`IDLE`, `WON`)	`CaroEngine` **(TS Ground Truth)**	Game Session	`CaroEngine`	React HUD (Observer)	Main Thread Controller
Canvas Scene Graph & Sprites	PixiJS Stage	Pixi Application	None	**PixiJS (WebGL)**	Main Thread RAF
User Interface Overlays & Modals	React Component State	React DOM Lifecycle	React UI Shell	**React Virtual DOM**	Main Thread
Search Tree & Transposition Table	AI Worker Thread	Search Duration	Alpha-Beta Module	None (Headless)	**Web Worker Thread**
Sound Effects & Audio Bus	Audio Controller	App Session	UI Event Dispatcher	Web Audio API	Main Thread

---

4. 10-Week Research Roadmap & Weekly Sequence Diagrams

All reports are authored under **Standard 00 (IMRaD & Evidence Specification)** and compiled into `.md`, `.docx`, and `.pdf` formats.

```
FINAL/
■■■ 00A_Master_Research_and_Evidence_Registry/     # Master Control Layer & Metric Dictionary
■■■ 00_De_Cuong_PDF/                               # Official University Syllabus
■■■ Week_01/                                       # Mathematical Foundations & Rule Profiling
■■■ Week_02/                                       # TypeScript Core Engine & Invariant Matrix
■■■ Week_03/                                       # Web UI, PixiJS Canvas & Coordinate Contract
■■■ Week_04/                                       # Four-Block Experimentation & Rust WASM
■■■ Week_05/                                       # Minimax Search & SearchSpaceMode Isolation
■■■ Week_06/                                       # Alpha-Beta Pruning & Synthetic Tree Tests
■■■ Week_07/                                       # Candidate Move Generation & Tactical Recall
■■■ Week_08/                                       # Dynamic Move Ordering & Cutoff Acceleration
■■■ Week_09/                                       # Zobrist Hashing & Transposition Tables
■■■ Week_10/                                       # Iterative Deepening & Bot Tournaments
■■■ 98_Thesis_Synthesis_Matrix/                    # Chapter Mapping & Final Claim Gate
■■■ 99_Reading_Index_and_References/               # Canonical Bibliography & Verification URLs
■■■ Google_Drive_Sync/                             # Consolidated Delivery Artifacts for GDrive
■■■ scripts/
    ■■■ build_docs.py                              # Document Compiler
```

4.1 Weekly Sequence Overview (Weeks 01 to 10)

Week 01 Sequence: Mathematical Foundations & Rule Contracts

```
sequenceDiagram
    autonumber
    actor Research as Researcher
    participant Math as Combinatorial Model
    participant Memory as L1 Cache Model
    participant Board as Board [u8; 225]

    Research->>Math: Compute State Space (3^225 ≈ 6.89e107) & Game Tree (10^105)
    Research->>Math: Verify First-Player Win Proof (Allis 1994)
    Research->>Memory: Verify 1D Array cache alignment (225 bytes < 4 L1 cache lines)
    Research->>Board: Initialize 4 Directional Vectors: (1,0), (0,1), (1,1), (1,-1)
    Research->>Board: Lock Rule Decision Table (CFG-RULE-v1) & Fixtures FIX-RULE-001..005
```

Week 02 Sequence: TypeScript Core Engine & Invariant Matrix

```
sequenceDiagram
    autonumber
    actor Player as Human / Bot
    participant Engine as CaroEngine (TS)
    participant Board as Board Buffer (Uint8Array)
    participant Stack as Move History Stack
    participant Suite as Vitest Test Suite

    Player->>Engine: makeMove(x, y)
    Engine->>Engine: Validate Invariants INV-01 to INV-07
    Engine->>Board: Mutate board[idx] = currentPlayer
    Engine->>Stack: Push Move Record
    Engine->>Engine: Localized Raycast Win Check (max 36 steps)
    Engine-->>Player: Return Result { status, winner }
    Suite->>Engine: Run 18 Vitest Unit Tests
    Suite-->>Engine: 18/18 Passed (42ms)
```

Week 03 Sequence: PixiJS Canvas Viewport & Coordinate Contract

```
sequenceDiagram
    autonumber
    actor User as Player Tap
    participant Pixi as PixiJS Canvas
    participant Map as Coordinate Mapper
    participant Guard as Concurrency Guard (isBusy)
    participant Engine as CaroEngine
    participant HUD as React HUD

    User->>Pixi: pointerdown(clientX, clientY)
    Pixi->>Map: screenToGrid(clientX, clientY)
    Map-->>Pixi: Discrete coordinates (x, y) within snap radius
    Pixi->>Guard: acquireLock()
    alt Mutex Busy
        Guard-->>Pixi: Drop tap (Event Uniqueness Contract)
    else Mutex Acquired
        Pixi->>Engine: makeMove(x, y)
        Pixi->>Pixi: Batch render stone sprite (Single WebGL draw call)
        Pixi->>HUD: Sync State Snapshot
        Pixi->>Guard: releaseLock()
    end
```

Week 04 Sequence: Four-Block Experimentation & Rust WASM

```
sequenceDiagram
    autonumber
    participant Runner as Experiment Runner
    participant Core as Rust WASM Core
    participant Scanner as 4-Direction Scanner
    participant Eval as Asymmetric Evaluator

    Runner->>Core: EXP-W4-A01: Verify Semantic Equivalence (TS == Rust)
    Core-->>Runner: Gate 1 PASS (100% agreement)
    Runner->>Scanner: EXP-W4-B01: Classify 7 Gomoku Shapes (CFG-PATTERN-v1)
    Scanner-->>Runner: Zero double-counting verified
    Runner->>Eval: EXP-W4-C01: Compute Heuristic (Attack - 1.15 * Defense)
    Runner->>Core: EXP-W4-D01: Benchmark Throughput (JS vs Rust WASM)
    Core-->>Runner: Rust WASM achieves 1.84M evals/sec (15.3x speedup)
```

Week 05 Sequence: Minimax Search & SearchSpaceMode Isolation

```
sequenceDiagram
    autonumber
    participant AI as AI Bot Controller
    participant MM as Minimax Search Engine
    participant Mode as SearchSpaceMode Controller
    participant Gen as Move Generator

    AI->>MM: minimax_search(depth = 3, space_mode)
    MM->>Mode: Check SearchSpaceMode
    alt FULL_LEGAL (MM-FULL-D3)
        Mode->>Gen: generate_all_legal_moves() -> 221 moves
        MM->>MM: Combinatorial explosion (> 10M nodes, 6.2s timeout!)
    else CANDIDATE_RESTRICTED (MM-CAND-v1-D3)
        Mode->>Gen: generate_candidate_moves(r = 2) -> 18 moves
        MM->>MM: Search completes: 6,175 nodes in 3.84ms
    end
    MM-->>AI: Return verified optimal move
```

Week 06 Sequence: Alpha-Beta Pruning & Synthetic Tree Tests

```
sequenceDiagram
    autonumber
    participant Suite as Equivalence Test Suite
    participant AB as Alpha-Beta Engine
    participant Synth as Synthetic Tree Oracle

    Suite->>Synth: Run SYN-TREE-01 to 04 (Debug cutoff semantics)
    Synth-->>Suite: All theoretical bounds and cutoffs verified
    Suite->>AB: EXP-W6-01: Run search_equivalence on Caro positions
    AB->>AB: Evaluate with fail-soft [alpha, beta] window
    AB-->>Suite: 3/3 Equivalence Tests PASS (92.9% nodes pruned at depth 4)
```

Week 07 Sequence: Candidate Move Generation & Tactical Recall

```
sequenceDiagram
    autonumber
    participant Gen as Candidate Generator (r = 2)
    participant Recall as Tactical Recall Suite
    participant Board as Board State

    Board->>Gen: Generate active candidates C(s)
    Gen-->>Board: Return candidate set (84% spatial reduction)
    Recall->>Gen: Test FIX-TACTICAL-001..025
    Gen-->>Recall: TacticalRecall = 1.00 (100% critical threats retained)
```

Week 08 Sequence: Dynamic Move Ordering & Cutoff Acceleration

```
sequenceDiagram
    autonumber
    participant AB as Alpha-Beta Engine
    participant Sorter as Move Ordering Sorter
    participant Eval as 1-Ply Tactical Pre-Evaluator

    AB->>Sorter: sort_candidates(candidates)
    Sorter->>Eval: Quick tactical evaluation per move
    Sorter-->>AB: Re-ordered moves (Best candidate examined first)
    AB->>AB: First-move cutoff ratio rises from 32.4% to 78.9%
    AB-->>AB: Depth 5 search accelerated 7.15x (44.8ms)
```

Week 09 Sequence: Zobrist Hashing & Transposition Table

```
sequenceDiagram
    autonumber
    participant Search as Alpha-Beta Search
    participant TT as 8MB Zobrist Transposition Table

    Search->>TT: probe(zobrist_hash)
    alt TT Hit (depth >= current_depth && bound valid)
        TT-->>Search: Return cached score and best move (Skip subtree!)
    else TT Miss
        Search->>Search: Recurse children
        Search->>TT: store(hash, depth, score, bound, best_move)
    end
    Note over Search,TT: Usable hit rate reaches 38.5% at depth 6
```

Week 10 Sequence: Iterative Deepening & Bot Tournament

```
sequenceDiagram
    autonumber
    actor Player as Human / Bot
    participant ID as Iterative Deepening Engine
    participant Timer as Deadline Monitor (500ms)
    participant Tournament as Round-Robin Harness

    Player->>ID: search_with_budget(500ms)
    ID->>ID: Depth 1 -> 2 -> 3 -> 4 -> 5 -> 6
    Timer-->>ID: Check deadline (0.0ms overshoot)
    ID-->>Player: Return deepest completed iteration (Depth 6)
    Tournament->>ID: Execute 6-Tier Round-Robin Tournament (100 games/pair)
    Tournament-->>Player: Tier 6 Master Bot achieves 96.0% overall win rate
```

---

5. Master Subtask Backlog (`ST-001` to `ST-056`)

Subtask ID	Subtask Description	Subsystem Scope	Evidence Status
ST-001	Create Master Registry (`00A`) and Universal ID schema	Control Layer	`VERIFIED` **`[A]`**
ST-002	Assign immutable IDs to all Research Questions	Control Layer	`VERIFIED` **`[A]`**
ST-003	Construct canonical Fixture Registry (`FIX-RULE`, `FIX-TACTICAL`, `FIX-HORIZON`)	Fixtures	`VERIFIED` **`[A]`**
ST-004	Construct Claim Registry with strict evidence tagging	Claims	`VERIFIED` **`[A]`**
ST-005	Construct Experiment Registry with variable isolation	Experiments	`VERIFIED` **`[A]`**
ST-006	Lock canonical RuleSet (`CFG-RULE-v1`) and decision table	Rules	`VERIFIED` **`[A]`**
ST-007	Codify System Metric Dictionary (`nodesVisited`, `cutoffCount`, etc.)	Metrics	`VERIFIED` **`[A]`**
ST-010	Map repository structure (`tducn110/caro` commit `cb6fd6f`)	Repository	`VERIFIED` **`[A]`**
ST-011	Verify current RuleSet implementation in TypeScript	Core Engine	`VERIFIED` **`[A]`**
ST-012	Map state ownership matrix and SSOT invariants	Architecture	`VERIFIED` **`[A]`**
ST-013	Map algorithmic search routines	AI Engine	`VERIFIED` **`[A]`**
ST-014	Map existing test suites (18 Vitest tests, 9 Cargo tests)	Tests	`VERIFIED` **`[A]`**
ST-020	Execute TypeScript baseline tests	TS Baseline	`VERIFIED` **`[A]`**
ST-021	Construct TS $\leftrightarrow$ Rust equivalence corpus	Equivalence	`VERIFIED` **`[A]`**
ST-024	Formally record `ADR-ARCH-01` for Web Worker + Rust WASM core	Architecture	`VERIFIED` **`[A]`**
ST-030	Lock 7-shape pattern taxonomy (`CFG-PATTERN-v1`)	Evaluation	`VERIFIED` **`[A]`**

ST-032	Calibrate heuristic weights with defense multiplier $\beta = 1.15$	Evaluation	VERIFIED [A]
ST-034	Implement `SearchSpaceMode` parameterization in Minimax	Minimax	VERIFIED [A]
ST-035	Build Minimax `FULL_LEGAL` vs `CANDIDATE_RESTRICTED` scaling corpus	Experiments	VERIFIED [A]
ST-036	Build Alpha-Beta search equivalence test suite (`search_equivalence.rs`)	Pruning	VERIFIED [A]
ST-040	Run candidate move reduction experiment (`EXP-W7-01`)	Candidates	PLANNED [D]
ST-041	Run tactical recall experiment (`EXP-W7-02`)	Candidates	PLANNED [D]
ST-042	Run dynamic move ordering benchmark (`EXP-W8-01`)	Ordering	PLANNED [D]
ST-043	Verify incremental Zobrist hash bitwise equality	Hashing	PLANNED [D]
ST-044	Verify Transposition Table bound semantics	Caching	PLANNED [D]
ST-045	Execute TT performance and hit rate benchmark (`EXP-W9-01`)	Caching	PLANNED [D]
ST-046	Verify Iterative Deepening deadline enforcement (`EXP-W10-01`)	Real-Time	PLANNED [D]
ST-050	Seal production configuration `CFG-FINAL-v1`	Production	PLANNED [D]
ST-052	Run multi-tier Bot-vs-Bot round-robin tournament (`EXP-W10-02`)	Tournament	PLANNED [D]
ST-056	Complete Thesis Synthesis Matrix and Chapter Mapping (`98`)	Thesis	VERIFIED [A]

---

6. Final Control Rule

From the formal establishment of this backbone forward:

$$\text{SPECIFICATION} \ne \text{IMPLEMENTATION EVIDENCE}$$ $$\text{IMPLEMENTATION} \ne \text{CORRECTNESS EVIDENCE}$$ $$\text{CORRECTNESS} \ne \text{PERFORMANCE EVIDENCE}$$ $$\text{PERFORMANCE DATA} \ne \text{GENERAL CONCLUSION}$$

Every layer must pass through its dedicated gate: $$\text{Define} \longrightarrow \text{Implement} \longrightarrow \text{Verify} \longrightarrow \text{Experiment} \longrightarrow \text{Interpret} \longrightarrow \text{Claim}$$